HeatWise — Technical Documentation

Version: 2.1 | **Date:** March 2026 | **Stack:** Next.js 16 · Prisma · SQLite/Postgres · Three.js · Capacitor

Table of Contents

- Architecture Overview
- Tech Stack
- Project Structure
- Database Schema
- Recommendation Engine
- Species Catalog System
- ML Integration
- API Reference
- Authentication
- AR & 3D Visualization
- Telemetry & Observability
- Android / Mobile
- Environment Variables
- Development Commands
- CI / Quality Gates
- Deployment

1. Architecture Overview

```
┌──────────────────────────────────────────────────────────────┐
│                        HEATWISE PLATFORM                     │
│                                                              │
│  ┌──────────────┐    ┌──────────────┐    ┌──────────────────┐ │
│  │  React PWA   │    │  Android     │    │   Admin Dashboard   │ │
│  │  (Next.js)   │    │  (Capacitor) │    │   /admin_analytics/ │ │
│  └──────┬───────┘    └──────┬───────┘    └──────────┬───────┘ │
│         │                   │                       │         │
│         └───────────────────┴───────────────────────┘         │
│                             │                                 │
│                     ┌───────▼──────┐                          │
│                     │  Next.js API    │  (pages/api/*)         │
│                     │  Route Layer    │                        │
│                     └──────┬──────┘                          │
│                            │                                  │
│         ┌──────────────────┼──────────────────┐               │
│         ▼                  ▼                  ▼               │
│  ┌──────────────┐    ┌────────────────┐   ┌──────────────┐    │
│  │  Prisma      │    │ Recommendation │   │  OpenAI API  │    │
│  │  ORM         │    │    Engine      │   │  (GPT-4V)    │    │
│  │  SQLite /    │    │  (TS + Python) │   │  Visualization│    │
│  │  Postgres    │    └────────────────┘   └──────────────┘    │
│  └──────────────┘                                             │
└──────────────────────────────────────────────────────────────┘
```

Design Principles

- **Offline-first fallbacks**: 3-layer recommendation fallback (ML → Catalog Hybrid → Rules-Only)
- **Immutable telemetry**: All events are append-only; no updates to telemetry rows
- **Progressive enhancement**: App works without ML; quality improves with ML online
- **Mobile-first**: Designed for 390px screens; deployed via Capacitor to Android

2. Tech Stack

Layer	Technology	Version	Purpose
Framework	Next.js (Pages Router)	16.x	Full-stack web + API
UI	React	18.x	Component tree
Styling	Inline JSX styles + CSS-in-JS	—	Theme-consistent UI
3D/AR	Three.js	latest	Garden models, AR overlay
Database (dev)	SQLite via Prisma	5.x	Local persistence
Database (prod)	PostgreSQL	14+	Production persistence

ORM	Prisma	5.x	Type-safe DB access
Auth	NextAuth.js	4.x	Phone OTP sessions
Animation	Framer Motion	10.x	Screen transitions
Mobile	Capacitor	5.x	Android WebView wrapper
AI	OpenAI GPT-4V	—	Garden visualization
ML Runtime	Python 3.11+	—	Plant recommendation scoring
Language	TypeScript + JSX	5.x	Type safety

3. Project Structure

```
heatwise/
├── pages/
│   ├── index.jsx              # Main app entry (renders HeatWiseApp)
│   └── api/
│       ├── phone-auth/        # OTP send + verify
│       ├── recommendations/   # generate, session, feedback, rollout
│       ├── reports/           # dossier, user-report, installer-summary
│       ├── installers/        # quote-request, assignment, outcomes
│       ├── geospatial/        # enrich-project, microclimate, exposure
│       ├── supply/            # species-availability, regional-readiness
│       ├── admin/             # metrics, experiments, rollout-monitor
│       ├── commercial/        # funnel, revenue, unit-economics
│       ├── governance/        # consent, policy-flags, retention
│       ├── followups/         # longitudinal re-measurement
│       └── insights/          # lesson-memory, segment-performance
│
├── components/
│   ├── HeatWiseApp.jsx        # 5000+ line monolithic app component
│   └── heatwise/
│       ├── theme.js           # Color tokens (T.green, T.orange, etc.)
│       ├── styles.js          # Global CSS string
│       ├── auth/              # PhoneLoginScreen, OTPVerification, CompleteProfile
│       ├── ui/                # Icon, BottomNav
│       └── feedback.js        # logRecommendationFeedback, fetchHeatReductionSummary
│
├── lib/
│   ├── recommendation/
│   │   ├── orchestrateLayoutRecommendations.ts   # Entry point
│   │   ├── catalogHybridFallback.ts              # Main scoring engine (37KB)
│   │   ├── rulesOnlyFallback.ts                  # Pure rules fallback
│   │   ├── mergeMlIntoLayoutRecommendations.ts   # ML result merger
│   │   ├── layoutEligibility.ts                  # Pre-filter eligibility
│   │   ├── buildRecommendationGenerateRequest..  # Request builder
│   │   └── testing/scenarios.ts                  # 7 regression scenarios
│   ├── species/
│   │   ├── speciesCatalogMapping.ts              # Alias → code mapping
│   │   └── resolveSpeciesCatalogCode.ts          # Runtime resolution
│   ├── ml/
│   │   ├── recommendationRuntimeTypes.ts         # Shared types
│   │   └── exportTelemetryPipeline.ts            # Training data export
│   ├── installerExport.ts                        # Installer data builder
│   ├── phoneOtp.ts                               # OTP logic
│   └── db.ts                                     # Prisma client singleton
│
├── prisma/
│   ├── schema.prisma          # 55+ models, 1874 lines
│   ├── seed.mjs               # Main seed runner
│   └── data/
│       └── species_catalog_seed.mjs   # 51 species definitions
│
├── ml/
│   ├── serving/               # Python serving scripts
│   ├── registry/              # Trained model artifacts
│   └── data/bootstrap/        # Species feature CSVs
│
├── data/
│   └── species/
│       └── species_catalog_mapping.v1.json   # Generated artifact (51 codes, 179 aliases)
```

```
│
├── src/
│   ├── ar/                     # AR overlay, spatial mapping
│   └── live-ar/                # Live camera AR tracking + geometry
│
├── models/
│   └── index.ts                # Core domain types
│
└── heatwise-native/            # Capacitor Android project
```

4. Database Schema

Core Models (55+ total)

```
User
├── id, name, email, phoneNumber, phoneVerified
├── profileCompleted, city, state, country
└── → projects[], telemetrySessions[], installOutcomes[]


Project
├── id, name, location, surfaceType, primaryGoal
├── area (Float), status
└── → analysis?, spaces[], recommendationRuns[]


SpeciesCatalog                          ← 51 seeded species
├── code (unique)                        e.g. "sedum", "vetiver"
├── displayName, scientificName, category
├── edible, flowering, petSafe
├── droughtTolerant, heatTolerant, lowMaintenance
├── minSunHours, maxSunHours
└── tagsJson


RecommendationTelemetrySession          ← immutable
├── id, userId, projectId
├── modelVersion, rulesVersion
├── generatedAt, source (ml/hybrid/rules)
└── → candidateSnapshots[]


RecommendationCandidateSnapshot         ← immutable
├── rank, score, speciesCodes[]
├── layoutType, estimatedCostInr
└── → telemetryEvents[]


RecommendationTelemetryEvent            ← append-only
├── eventType (view/select/dismiss/feedback)
├── screenName, metadata (JSON)
└── createdAt


InstallerQuoteRequest
├── userId, projectId, region
├── contactMethod, timeline
└── → InstallerInstallJob?


RecommendationDossier
├── dossierId, recommendationSessionId
├── dossierType, selectedCandidateSnapshotId
└── → userReport?
```

Key Relationships

```
User ──────────────────────► Project (1:many)
Project ────────────────────► RecommendationRun (1:many)
RecommendationRun ──────────► RecommendationTelemetrySession (1:1)
TelemetrySession ───────────► CandidateSnapshots[] (1:many)
CandidateSnapshot ──────────► TelemetryEvents[] (1:many, append-only)
```

5. Recommendation Engine

5.1 Entry Point

```
POST /api/recommendations/generate
  └─► orchestrateLayoutRecommendations(request)
```

5.2 Three-Layer Fallback Architecture

```
Request
   │
   ├─► 1. Python ML Serving (full_ml)
   │       ml/serving/ scripts via child_process
   │       Returns: scored plant list with ML confidence
   │       Failure triggers → layer 2
   │
   ├─► 2. Catalog Hybrid Fallback (partial_ml / catalog_hybrid_ts)
   │       lib/recommendation/catalogHybridFallback.ts  [37KB]
   │       • heuristicRankScore() — 4 sub-score blend
   │       • scarceWaterPriorityFactor() — xeric boost
   │       • petSafeRankingNudge() — pet-safe scenario nudge
   │       • blendScores() — weighted combination
   │       Failure triggers → layer 3
   │
   └─► 3. Rules-Only Fallback (rules_only)
           lib/recommendation/rulesOnlyFallback.ts
           Pure TypeScript, zero dependencies
           Always available
```

5.3 Scoring in Catalog Hybrid

```
heuristicRankScore(species, project, environment, preferences):
  rulePrior    = base trait match (drought, sun, wind)
  feasibility  = budget + load capacity + container fit
  heat         = coolingScore + shade contribution
  ranking      = maintenance gap + pollinator bonus


  // Scenario-specific adjustments
  if (water_availability === 'scarce'):
    heat *= scarceWaterPriorityFactor(species)  // up to 1.30x for xeric plants


  if (child_pet_safe_required):
    ranking *= petSafeRankingNudge(species)     // 0.80-1.08x targeted nudge


  final = blendScores(rulePrior, feasibility, heat, ranking, weights)
```

5.4 Layout Eligibility

```
layoutEligibility.ts checks:
  • space_kind (balcony / terrace / rooftop)
  • width_m × length_m → area eligibility
  • floor_level → wind exposure rules
  • load_capacity_level → plant weight constraints
```

5.5 HeatReductionSummary

Calculated after scoring:

```
estimatedDropC = f(plantCoverageRatio, shadeCoverageRatio, reflectiveCoverageRatio)
confidence     = 'high' | 'medium' | 'low'  (from ML layer)
drivers[]      = human-readable cooling factor list
```

6. Species Catalog System

6.1 Catalog Size

- **51 species** seeded (as of March 2026)
- **179 aliases** in mapping artifact

6.2 Species Categories

Category	Count	Examples
HERB	10	tulsi_holy, mint, coriander, brahmi, fenugreek
ORNAMENTAL	8	bougainvillea, marigold, portulaca, crossandra

VEGETABLE	8	okra, cherry_tomato, bitter_gourd, malabar_spinach
SUCCULENT	7	sedum, prickly_pear, adenium, aloe_vera
FOLIAGE	7	pothos, snake_plant, dracaena_marginata, coleus
GRASS	4	vetiver, lemongrass, bamboo_dwarf, lemongrass_dense
SHRUB	4	plumeria, jasmine_mogra, duranta, henna
CLIMBER	3	ficus_pumila, morning_glory, malabar_spinach

6.3 Species Identity Pipeline

```
App input (string)
  └► resolveSpeciesCatalogCode(alias)
        └► species_catalog_mapping.v1.json  (generated artifact)
              └► canonical code  e.g. "vetiver"
                    └► SpeciesCatalog DB row
```

6.4 Regenerate Mapping

```
npm run gen:species-mapping    # Reads seed + DB → writes JSON artifact
npm run check:species-mapping  # Verify in CI
```

7. ML Integration

7.1 Python Serving

```
ml/
├── serving/          Python scripts called by Node
├── registry/         Trained model artifacts (.pkl / ONNX)
└── data/bootstrap/   species_features.csv (bootstrap data)
```

7.2 Environment Config

```
HEATWISE_ML_CWD=./ml
HEATWISE_REGISTRY_DIR=./ml/registry
HEATWISE_SPECIES_CSV=./ml/data/bootstrap/sample_outputs/demo_pack/species_features.csv
```

7.3 Node → Python Bridge

```
// lib/ml/mlServing.ts (simplified)
const child = spawn('python3', ['serving/score.py'], {
  cwd: process.env.HEATWISE_ML_CWD,
  stdio: ['pipe','pipe','pipe'],
});
child.stdin.write(JSON.stringify(request));
// Parse stdout as scored plant JSON
// On non-zero exit or empty stdout → trigger catalog hybrid fallback
```

7.4 Training Data Export

```
// lib/ml/exportTelemetryPipeline.ts
// Reads RecommendationTelemetryEvent rows with eventType='install_confirmed'
// Exports to CSV for retraining
```

8. API Reference

Authentication

Method	Endpoint	Description
POST	`/api/phone-auth/send-otp`	Send OTP to phone number
POST	`/api/phone-auth/verify-otp`	Verify OTP, create session

Recommendations

Method	Endpoint	Description
POST	`/api/recommendations/generate`	**Main endpoint** — generate plan
POST	`/api/recommendations/create-session`	Register telemetry session
POST	`/api/recommendations/submit-feedback`	User rating/feedback
POST	`/api/recommendations/mark-selected`	Log candidate selection
GET	`/api/recommendations/rollout-status`	ML rollout percentage
POST	`/api/recommendations/submit-install-outcome`	Post-install result

Projects

Method	Endpoint	Description

GET	`/api/projects`	List user projects
POST	`/api/projects`	Create project
GET	`/api/projects/[id]`	Get project detail
PUT	`/api/projects/[id]`	Update project

Reports

Method	Endpoint	Description
POST	`/api/reports/recommendation-dossier`	Create dossier
GET	`/api/reports/user-report`	Fetch homeowner summary
GET	`/api/reports/installer-summary`	Installer-facing summary

Installers

Method	Endpoint	Description
POST	`/api/installers/quote-request`	Submit RFQ
POST	`/api/installers/assignment`	Assign installer to job
GET	`/api/installers/outcomes`	Job outcome records

Geospatial

Method	Endpoint	Description
POST	`/api/geospatial/enrich-project`	Auto-fill location metadata
GET	`/api/geospatial/microclimate`	Local temperature data
GET	`/api/geospatial/site-exposure`	Sun/wind exposure by coordinates

Supply Chain

Method	Endpoint	Description
GET	`/api/supply/species-availability`	Regional plant availability
GET	`/api/supply/seasonal-windows`	Best planting months

GET	`/api/supply/regional-readiness`	Installer capacity by region

Admin

Method	Endpoint	Description
GET	`/api/admin/metrics-overview`	Platform KPIs
GET	`/api/admin/rollout-monitor`	ML rollout health
GET	`/api/admin/recommendation-funnel`	Conversion funnel

AI Visualization

Method	Endpoint	Description
POST	`/api/generate-garden-visual`	OpenAI GPT-4V garden image
POST	`/api/generate-layout`	Layout SVG generation

9. Authentication

Flow

```
1. POST /api/phone-auth/send-otp
   • Normalizes phone number (+91XXXXXXXXXX format)
   • Generates 6-digit OTP
   • bcryptjs.hash(otp, 10) → stored in PhoneOtp table
   • In dev: OTP logged to console + returned in response (debugOtp field)
   • In prod: send via SMS provider (Twilio/etc — wire in sendOtpSms())

 • POST /api/phone-auth/verify-otp



 • bcryptjs.compare(input, storedHash)
   • Max 8 attempts before lockout
   • On success: upsert User, create NextAuth session
   • Session stored in signed JWT (NEXTAUTH_SECRET)
```

Session Access

```
import { getServerSession } from "next-auth";
const session = await getServerSession(req, res, authOptions);
const userId = session?.user?.id;
```

Dev OTP Mode

```
HEATWISE_DEV_OTP=true    # OTP printed to server console (no SMS)
```

10. AR & 3D Visualization

Three.js Scenes (in HeatWiseApp.jsx)

Component	Screen	Description
`MegaCityScene`	Splash	120-building city with heat shimmer + scan beam
`HoloGlobe`	Home hero	Temperature globe with orbiting satellites
`NeuralBrain`	Analysis	Neural network nodes with signal particles
`GardenScene3D`	**Result hero**	Species-specific 3D rooftop garden
`DataOrb`	Background	Particle sphere for settings/save screens
`RooftopTransform`	(legacy)	Simple roof with generic trees

GardenScene3D Plant Types

```
succulent  → low dome + radial spines
grass      → 16-blade tuft with sway animation
herb       → compact leaf cluster on stem
climber    → trellis frame + vine leaves
shrub      → multi-tier sphere stack
vegetable  → leaf planes + fruit dot particles
ornamental → flower bush with colored petal spheres
foliage    → broad sphere canopy
```

Live AR (src/live-ar/)

```
LiveARMeasurementScreen
├── Camera stream (getUserMedia)
├── Three.js canvas overlay
├── Tracking: perspective math + geometry
├── ManualMeasurement fallback
└── Output: widthM, lengthM → photoSession
```

11. Telemetry & Observability

Immutable Append-Only Design

```
RecommendationTelemetrySession     — one per generate() call
  ├── modelVersion                 — string, frozen at creation
  ├── rulesVersion                 — string, frozen at creation
  └── RecommendationCandidateSnapshot[]  — one per candidate


RecommendationTelemetryEvent       — append-only user signals
  ├── recommendation_run_viewed
  ├── candidate_selected
  ├── candidate_dismissed
  ├── report_opened
  └── install_confirmed


RecommendationRuntimeObservation   — rollout metrics
  ├── source (ml/hybrid/rules)
  ├── mlErrors[]
  └── guardrailFailures[]
```

Key Rule: Never update existing telemetry rows

Evaluation / Regression Testing

```
npm run test:recommendation     # Runs 7 scenarios against live DB
npm run report:recommendation-eval   # Generates MD + CSV report
```

7 Regression Scenarios:

- sunny_balcony_low_maintenance
- hot_terrace_cooling
- shaded_balcony_aesthetic
- pet_safe_home_garden
- edible_herb_setup
- windy_highrise_balcony
- water_scarce_terrace

12. Android / Mobile

Setup

```
npm run dev:android      # Start dev server for Android
npm run android:sync     # npx cap sync android
npm run android:studio   # Open Android Studio
```

WebView Configuration

```
CAP_SERVER_URL=http://localhost:3000      # USB / emulator
<h1>CAP_SERVER_URL=http://10.0.2.2:3000    # Android emulator loopback</h1>
<h1>CAP_SERVER_URL=http://YOUR_LAN_IP:3000 # Physical device on Wi-Fi</h1>
```

ADB for USB development

```
adb reverse tcp:3000 tcp:3000
```

Allowed Origins (next.config.mjs)

- `http://localhost:3000`
- `http://10.0.2.2:3000` (emulator)
- Custom via `HEATWISE_DEV_EXTRA_ORIGINS`

13. Environment Variables

```
<h1>Database</h1>
DATABASE_URL="file:./prisma/dev.db"          # SQLite dev
<h1>DATABASE_URL="postgresql://..."            # Postgres prod</h1>

<h1>Auth</h1>
NEXTAUTH_URL=http://localhost:3000
NEXTAUTH_SECRET=your-secret-here             # Min 32 chars in prod

<h1>AI</h1>
OPENAI_API_KEY=sk-...                        # For garden visualization

<h1>Dev flags</h1>
HEATWISE_DEV_OTP=false                       # true = log OTP to console
HEATWISE_ADMIN_EMAILS=admin@yourco.com       # Comma-separated

<h1>Android</h1>
CAP_SERVER_URL=http://localhost:3000
HEATWISE_DEV_EXTRA_ORIGINS=                  # Extra CORS origins

<h1>ML</h1>
HEATWISE_ML_CWD=./ml
HEATWISE_REGISTRY_DIR=./ml/registry
HEATWISE_SPECIES_CSV=./ml/data/bootstrap/sample_outputs/demo_pack/species_features.csv
```

14. Development Commands

```
<h1>Start</h1>
npm run dev                      # Next.js dev server → localhost:3000

<h1>Code Quality</h1>
npm run lint                     # ESLint
npm run typecheck                # tsc --noEmit

<h1>Database</h1>
npx prisma migrate deploy        # Apply schema migrations
npm run db:seed                  # Seed database (51 species + fixtures)
npm run db:migrate               # Run pending migrations

<h1>Species System</h1>
npm run gen:species-mapping      # Regenerate mapping artifact (commit result)
npm run check:species-mapping    # Verify mapping is up-to-date (CI gate)

<h1>Testing</h1>
npm run test:recommendation      # Integration tests (needs seeded DB)
npm run ml:serving-smoke         # ML serving smoke test

<h1>Reports</h1>
npm run report:recommendation-eval   # Run all 7 scenarios → MD/CSV report

<h1>Android</h1>
npm run dev:android              # Dev server with Android flags
npm run android:sync             # Sync Capacitor
npm run android:studio           # Open Android Studio
```

15. CI / Quality Gates

On every push/PR to `main`:

- **`npm run check:species-mapping`** — Verifies `species_catalog_mapping.v1.json` is committed and matches the seed source of truth
- **`npm run typecheck`** — TypeScript compilation (`tsc --noEmit`)

Local pre-commit checklist

```
npm run typecheck            # No TS errors
npm run check:species-mapping    # Mapping artifact up-to-date
npm run test:recommendation  # All 7 scenarios pass
```

16. Deployment

Production Requirements

- Node.js 20+
- PostgreSQL 14+
- Python 3.11+ (for ML serving)
- Environment variables configured (see §13)

Database Migration

```
DATABASE_URL="postgresql://..." npx prisma migrate deploy
DATABASE_URL="postgresql://..." npm run db:seed
```

ML Registry

Mount `ml/registry/` as a persistent volume. Models load from `HEATWISE_REGISTRY_DIR`.

Key Production Changes vs Dev

Setting	Dev	Production
`DATABASE_URL`	SQLite `file:./prisma/dev.db`	PostgreSQL connection string
`HEATWISE_DEV_OTP`	`true`	`false` (or omit)
`NEXTAUTH_SECRET`	Any string	Random 32+ char secret
`OPENAI_API_KEY`	Optional	Required for AI visuals
SMS Provider	Console output	Wire `sendOtpSms()` in `lib/phoneOtp.ts`

Appendix: Core Domain Types

```
// models/index.ts (key interfaces)

interface Plant {
  id: string;
  name: string;
  type: 'succulent'|'grass'|'herb'|'climber'|'shrub'|'vegetable'|'ornamental'|'foliage';
  waterNeeds: 'low'|'medium'|'high';
  sunRequirement: Array<'full'|'partial'|'shade'>;
  windTolerance: 'low'|'medium'|'high';
  coolingScore: number;           // 0-10
  speciesCatalogCode?: string;    // e.g. "sedum"
  edible?: boolean;
  petSafe?: boolean;
}

interface Candidate {
  scoredPlants: ScoredPlant[];
  layoutType: LayoutType;
  estimatedCostInr: number;
  heatReductionSummary: HeatReductionSummary;
}

interface HeatReductionSummary {
  estimatedDropC: number;
  plantCoverageRatio: number;
  shadeCoverageRatio: number;
  reflectiveCoverageRatio: number;
  confidence: 'high'|'medium'|'low';
  drivers: string[];
}

enum SpaceType { balcony, terrace, rooftop }
enum SunExposure { full, partial, shade }
enum WindLevel { low, medium, high }
enum MaintenanceLevel { minimal, low, moderate, high }
```

Maintained by HeatWise Engineering Team. For questions: open a GitHub issue.